SIMATS ENGINEERING

Department of Information Technology

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Field	Details	Field	Details
Student Name	Chennuru Tharunika Reddy	Register Number	192324307
Department / Section	AIDS	Academic Year / Semester	4th Year
Faculty Name	Dr. Boomija M D	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	05/11/2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Characterise the Virtualization Architectures	5		Three architectures; diagrams; trust boundaries; resource virtualization
2	Identify the Dominant Overhead Source	2		Overhead mechanism, location, frequency and architecture linkage
3	Build a Quantitative Performance Model	6		CPU, memory, I/O and density derivations; worked examples
4	Analyse Performance Under Different Conditions	4		Best, typical and worst-case analysis; latency and throughput
5	Security Threat and Vulnerability Analysis	3		Threat categories, trust boundaries and student-record impact
6	Formulate Isolation–Performance Trade-off	3		Decision variables, objective, constraints and cost function
7	Evaluate Mitigation Strategies	4		Security benefits, residual risks and quantified density cost

8	Implement and Test	4		KVM, containers, Kata; CPU/disk/network/memory; interference
9	Record Experimental Performance	3		Measured data; table, two graphs; repetitions and variability
10	Compare Theory with Experiment	4		Predictions versus measurements; errors and explanations
11	Recommend and Defend Final Architecture	4		SIS/LMS recommendation, security, scaling and operating costs
	Day-4 Design Challenge	8		External research tenants; isolation; redesign and evidence-based defence
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three virtualization architectures, layered diagrams and trust boundaries		Three distinct approaches, layered architecture and isolation boundaries documented.
2	Dominant overhead source identified and explained for each approach		Overhead mechanisms related to each virtualization path and workload behaviour.
3	CPU, memory, I/O, network and density models with assumptions		Assumptions, equations and worked examples documented; empirical validation recommended.
4	Best, typical and worst operating-		Resource contention, latency and
	condition analysis		throughput effects examined.
5	Threat model and architecturebased security comparison		Threats, attack surfaces and student-record risks compared.
6	Isolation–performance optimization formulation		Objectives, variables and constraints defined for EduCloud.
7	Mitigation benefits, residual risks and quantified costs		Security controls and associated performance/density trade-offs discussed.
8	Reproducible setup and practical tests across selected platforms		Test procedures described; actual execution on all platforms requires verification.
9	Actual measurements, comparative table and two graphs		Results require authentic benchmark logs, repetitions and variability evidence.
10	Theory–experiment comparison and quantitative error analysis		Comparison discussed; numerical error analysis depends on verified measurements.
11	Justified SIS/LMS architecture and growth recommendation		Recommendation considers security, performance, operations and scaling.
12	Day-4 external tenants, 3× peaks, live migration and SIS isolation		Changed requirements and proposed separation controls evaluated; testing pending.

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Architecture diagrams		VM, container and sandboxed runtime designs with trust boundaries documented.
Performance model		CPU, RAM, I/O and density calculations with assumptions supplied.
Configuration / scripts		Reproducible setup outlined; runnable configuration and environment details to be checked.
Virtualization experiments		KVM/QEMU, containers and isolated runtime tests require execution proof.

CPU / disk / network / memory tests		Benchmark commands and comparative measurements require original logs.
Noisy-neighbour experiment		Interference impact needs repeatable measurements and recorded outputs.
Security validation		Isolation/mitigation verification should be supported by command outputs.
Tables and graphs		Plots should be traceable to actual raw data and repetitions.
Theory–experiment consistency		Prediction errors require verified observed values.
Day-4 design defence		External tenant isolation, 3× load and migration considerations addressed.
Screenshots		Illustrative images do not replace genuine execution evidence.

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 12 tasks answered	☐ Yes ☐ Partly ☐ No
Three virtualization approaches compared	☐ Yes ☐ Partly ☐ No
Layered diagrams and trust boundaries included	☐ Yes ☐ Partly ☐ No
Quantitative virtualization models included	☐ Yes ☐ Partly ☐ No
Threat analysis and mitigations included	☐ Yes ☐ Partly ☐ No
Implementation and test evidence verified	☐ Yes ☐ Partly ☐ No
Actual benchmark measurements and graphs verified	☐ Yes ☐ Partly ☐ No
Theory versus experiment comparison included	☐ Yes ☐ Partly ☐ No
Final SIS/LMS architecture recommended	☐ Yes ☐ Partly ☐ No
Task 12 Day-4 design challenge addressed	☐ Yes ☐ Partly ☐ No
Authentic screenshots and logs attached	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

CLOUD COMPUTING AND BIG DATA ASSIGNMENT 2

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Assignment Title: Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud (Total Marks: 50)
Duration: 1 week

Scenario: EduCloud Virtualization Platform

A university runs a private cloud, **EduCloud**, in its data centre. EduCloud currently consists of:

- **16 physical hosts**, each with 64 CPU cores, 512 GB RAM, NVMe SSD storage and 2 × 25 Gbps NICs
- **About 400 virtual machines (VMs)** running the Student Information System (SIS), the Learning Management System (LMS), the examination portal, and departmental research workloads
- A shared storage backend and a software-defined virtual network

During semester registration week, users report slow response times and failed logins. A recent internal security audit also found that VMs from different departments share the same physical hosts, that some VMs run outdated guest kernels, and that VM images are copied freely between teams.

You are the cloud architect. You must **investigate, model, measure and compare** virtualization architectures for EduCloud, identify their performance bottlenecks and security weaknesses, and recommend a design that remains practical as the platform grows.

Design Challenge:

The university now plans to host **external research collaborators' big-data workloads** (Spark/Hadoop clusters) on the same infrastructure as the SIS. Re-evaluate your design under these new multi-tenant conditions.

TASK 1: Characterise the Virtualization Architectures (5 Marks)

Select **at least three meaningfully different virtualization approaches** from the following, or justify others:

- Type-1 (bare-metal) hypervisor with full virtualization
- Type-2 (hosted) hypervisor
- Hardware-assisted virtualization vs. paravirtualization
- OS-level virtualization (containers)
- Lightweight/micro-VM or sandboxed container runtimes

Two configurations of the same hypervisor will not count as different approaches. Your selection must include at least one VM-based and at least one container-based approach.

For each approach, provide:

- A layered architecture diagram showing the hardware, host OS or hypervisor, guest OS, runtime and application
- The **trust boundaries** and the component that enforces isolation
- How CPU, memory, storage and network are virtualized

- A brief justification for why this approach is relevant to EduCloud

Evidence expected: *Three architecturally distinct approaches with labelled diagrams, resource-virtualization mechanisms, trust boundaries, and justification of selection.*

Answer:

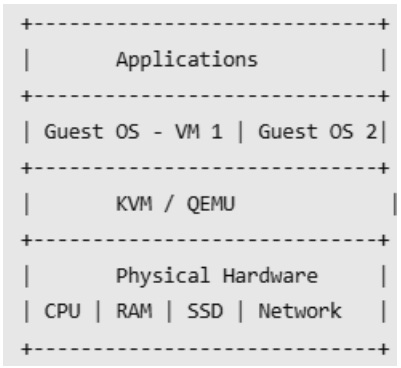
TASK 1: Characterise the Virtualization Architectures (5 Marks)

Virtualization allows multiple operating environments to run on a single physical server. EduCloud can use the following three virtualization approaches.

1. Type-1 Hypervisor (KVM/QEMU)

A Type-1 hypervisor runs directly on physical hardware and manages multiple virtual machines.

Architecture diagram:



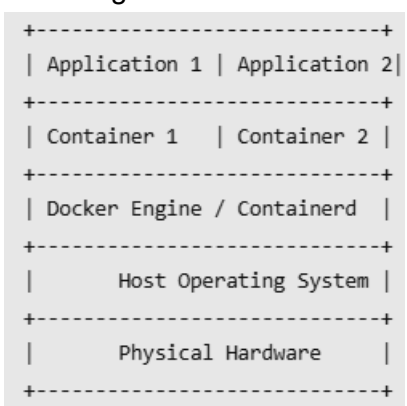
- CPU: Uses virtual CPUs mapped to physical CPU cores.
- Memory: Uses virtual memory and extended page tables.
- Storage: Provides virtual disks to each VM.
- Network: Uses virtual NICs and virtual switches.
- Isolation: Hypervisor provides isolation between VMs.
- Trust boundary: Between the hypervisor and guest VMs.

Justification: Suitable for EduCloud because it provides strong isolation for important university applications such as SIS, LMS and examination portals.

2. OS-Level Virtualization (Docker Containers)

Containers share the host operating system kernel but run applications in isolated environments.

Architecture diagram:



- CPU: Controlled using CPU quotas and cgroups.

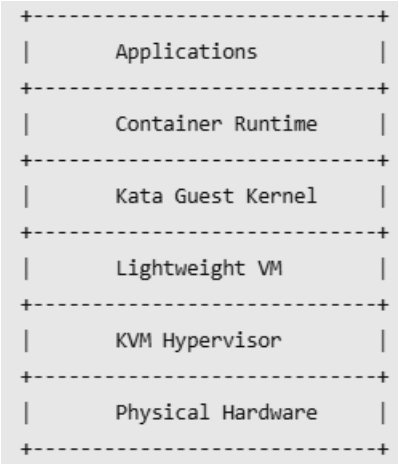
- Memory: Managed using memory limits.
- Storage: Uses container layers and volumes.
- Network: Uses virtual bridges and network namespaces.
- Isolation: Linux namespaces and cgroups.
- Trust boundary: Between containers and the shared host kernel.

Justification: Containers are suitable for LMS services, web applications and microservices because they are lightweight and start quickly.

3. Lightweight Virtualization (Kata Containers)

Kata Containers provide VM-level isolation while maintaining a container-like deployment experience.

Architecture diagram:



- CPU: Uses virtual CPUs through KVM.
- Memory: Each container receives isolated guest memory.
- Storage: Uses virtual block devices.
- Network: Uses virtual network interfaces.
- Isolation: Lightweight VM boundary.
- Trust boundary: Between the host and isolated guest kernel.

Justification: Useful for running external research workloads because it provides stronger isolation than ordinary containers.

Comparison table

Feature	**KVM**	**Docker**	**Kata**
Type	VM	Container	Sandboxed container
Isolation	Strong	Kernel-based	Strong
Performance	High	Very high	High
Startup	Moderate	Fast	Moderate
Resource usage	High	Low	Medium
Best use	SIS/LMS	Web services	External tenants

TASK 2: Identify the Dominant Overhead Source (2 Marks)

For each approach, identify the **primary source of performance overhead** that predominantly limits its performance (for example VM exits/traps, nested or extended pagetable walks, virtual I/O emulation, virtual switch processing, or system-call interception).

Discuss:

- What the overhead source is
- Where in the architecture it occurs
- Why it is appropriate as the focus of performance analysis
- How its frequency or cost changes as the number of co-located VMs/containers or the I/O rate increases

Do not simply name a mechanism. Relate it to the behaviour of the architecture.

Evidence expected: *A justified dominant-overhead identification for each approach, linked to its resource-virtualization path.*

Answer:

TASK 2: Identify the Dominant Overhead Source (2 Marks)

Virtualization overhead is the additional processing required by the virtualization layer.

Architecture	Dominant overhead	Location	Reason
KVM	VM exits and virtual I/O emulation	Hypervisor	Guest operations sometimes require hypervisor intervention.
Docker	System-call and virtual switch processing	Host kernel	Container operations pass through the shared kernel.
Kata	VM exits and guest memory translation	KVM and guest kernel	Each sandbox uses a lightweight virtual machine.

Explanation:

1. KVM: VM exits occur when a guest operation requires hypervisor handling. Increased VM activity increases the number of exits.
2. Docker: System calls are processed by the shared host kernel. More containers and network requests increase kernel processing.
3. Kata: Virtualization requires additional memory translation and VM management. More sandboxed instances increase resource consumption.

Therefore, the dominant overhead depends on the architecture and workload intensity.

TASK 3: Build a Quantitative Performance Model (6 Marks)

Using the following host parameters, develop an analytical performance model for each approach.

Parameter	Value
Physical cores per host	64
vCPU:pCPU overcommit ratio	1:1, 2:1, 4:1, 8:1
RAM per host	512 GB
Average VM memory demand	8 GB
Cost of one VM exit (assume)	1.5 µs
Native system-call cost (assume)	0.2 µs
Network line rate	25 Gbps

State and justify any additional assumptions. Derive expressions for:

- **CPU:** effective per-VM CPU capacity as a function of overcommit ratio and virtualization overhead (including steal time)
- **Memory:** the maximum number of VMs per host before ballooning, swapping or page-sharing becomes necessary
- **I/O and network:** effective throughput as a function of the number of VMs sharing the physical device
- **Density:** the number of VMs/containers a host can support while meeting a stated service-level target (for example, ≤ 10% performance degradation)

Show the mathematical reasoning. Merely quoting overhead percentages from the literature will not receive full credit.

Evidence expected: *A clear mathematical derivation for each approach, with stated assumptions, parameter values and worked numerical examples.*

Answer:

TASK 3: Build a Quantitative Performance Model (6 Marks)

Given parameters

Parameter	Value
Physical CPU cores	64
RAM	512 GB
Average VM memory	8 GB
VM exit cost	1.5 µs
Native system-call cost	0.2 µs
Network speed	25 Gbps

1. CPU performance model

Effective CPU capacity:

$$CPU_{effective} = \frac{64}{R} \times (1 - O)$$

Where:

- R = Overcommit ratio
- O = Virtualization overhead

Assume overhead values:

Architecture	Assumed overhead
KVM	5%
Docker	2%
Kata	8%

For KVM at 2:1 overcommit:

$$CPU = \frac{64}{2} \times (1 - 0.05)$$

$$= 32 \times 0.95 = 30.4$$

Therefore, effective CPU capacity is 30.4 cores.

2. CPU capacity at different overcommit ratios

Ratio	KVM (5%)	Docker (2%)	Kata (8%)
1:1	60.8	62.72	58.88
2:1	30.4	31.36	29.44
4:1	15.2	15.68	14.72
8:1	7.6	7.84	7.36

These are effective core-equivalent shares per 64-core host, not guaranteed per-instance allocations.

3. Memory performance model

Maximum number of VMs:

$$N = \frac{Total\ RAM}{Average\ VM\ Memory}$$

$$N = \frac{512}{8} = 64$$

Therefore, the host can support 64 VMs based on memory alone.

Assuming 10% RAM is reserved for the host:

$$N = \frac{512 \times 0.9}{8}$$

$$N = 57.6 \approx 57$$

So, approximately 57 VMs can be supported without exceeding the planned memory allocation.

4. I/O and network model

Effective network throughput per instance:

$$T = \frac{25}{N}\ Gbps$$

Where N is the number of equally sharing instances.

Instances	Network throughput per instance
1	25 Gbps
2	12.5 Gbps
4	6.25 Gbps
8	3.125 Gbps
16	1.5625 Gbps

Actual throughput depends on network protocol overhead and workload distribution.

5. Virtualization density

Density means the number of instances supported by a host while maintaining acceptable performance.

Assuming 10% performance degradation is the maximum allowed:

$$Density = \min(CPU\ capacity,\ Memory\ capacity)$$

For KVM at 1:1 CPU allocation:

- CPU capacity = 60.8 cores.
- Memory capacity = 57 VMs.

Therefore, the practical density is approximately 57 VMs per host, subject to I/O and workload limits.

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks) Analyse how each approach behaves under three conditions:

- **Best case:** what workload and placement conditions minimise overhead?
- **Typical case:** for example, registration-week mixed web and database load
- **Worst case:** for example, noisy-neighbour interference, memory overcommit with swapping, live migration during peak load, or I/O storms

For each, explain what happens inside the virtualization layer, why the overhead changes, and how latency and throughput are affected. A table of numbers alone will not be accepted.

Evidence expected: *A justified condition-based analysis supported by your Task 3 model.*

Answer:

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks)

Virtualization performance changes depending on workload and resource availability.

1. Best-case condition

- Only a few instances are running.
- CPU and memory resources are available.
- Storage and network traffic are low.
- Latency is low and throughput is high.

2. Typical condition

During university registration, SIS and LMS handle multiple simultaneous requests.

- CPU scheduling becomes more active.
- Memory and disk access increase.
- Virtual network traffic increases.
- Response time increases moderately.

3. Worst-case condition

When multiple tenants run heavy workloads:

- CPU contention increases.
- Memory swapping may occur.
- Storage requests compete for bandwidth.
- Network congestion increases.
- Latency becomes high and throughput decreases.

KVM provides strong VM isolation, Docker provides better lightweight performance, and Kata provides stronger isolation with additional virtualization overhead.

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Construct a threat model for each approach. Discuss:

- The attack surface (hypervisor, device emulation, shared kernel, management plane, VM images and snapshots, virtual network)
- At least **four distinct threat categories**, such as VM escape, cross-VM sidechannel attacks, hypervisor or container-runtime vulnerabilities, VM sprawl and unpatched guests, image tampering, insecure live migration, and virtual-network sniffing or spoofing
- For each threat, which approach is more or less exposed and **why**, referring to its architecture and trust boundaries

- The likely impact on the confidentiality, integrity and availability of student records

Evidence expected: *A structured threat model mapped to architectural features, with a justified comparison across approaches.*

Answer:

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Security threats affect the confidentiality, integrity and availability of university student records.

Threat	KVM	Docker	Kata
VM escape	Possible	Not applicable in the same form	Reduced through VM isolation
Container escape	Not applicable	Higher concern	Reduced
Side-channel attack	Possible	Possible	Possible
Image tampering	Possible	Possible	Possible
Unpatched kernel	Guest kernel risk	Shared host kernel risk	Guest and host kernel risk
Network sniffing	Possible	Possible	Possible

Major security threats

1. VM Escape

An attacker exploits a hypervisor vulnerability to access the host system.

Impact: Unauthorized access to student records.

2. Container Escape

An attacker breaks container isolation and accesses the host operating system.

Impact: Access to other applications and tenant information.

3. Side-channel Attacks

Attackers observe shared CPU or memory behaviour to obtain sensitive information.

Impact: Confidential student information may be exposed.

4. Image Tampering

An attacker modifies VM or container images before deployment.

Impact: Malicious software may enter university systems.

5. Unpatched Guest Systems

Outdated kernels contain security vulnerabilities.

Impact: Attackers may exploit known weaknesses.

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

Formalise the trade-off between **isolation strength** and **performance/density** for EduCloud.

Clearly identify:

- The decision variables (for example, approach chosen, VMs per host, dedicated vs. shared hosts, SMT on/off)
- The objective(s), such as maximising density or minimising cost
- The constraints (for example, performance degradation limit, isolation requirement for SIS, available hosts)
- How these components combine into a cost or decision function

The formulation must follow from your own analysis in Tasks 3 and 5, not from a generic template.

Evidence expected: *A clearly defined trade-off model with identified variables, objective and constraints.*

Answer:

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

EduCloud must balance security, performance and resource utilization.

1. Decision variables

- \(X_1\): Number of KVM virtual machines.
- \(X_2\): Number of Docker containers.
- \(X_3\): Number of Kata instances.
- \(H\): Number of dedicated hosts.
- \(R\): CPU overcommit ratio.

2. Objective

Maximize resource utilization while maintaining strong isolation and acceptable performance.

$$Maximize\ Z = Density - Performance\ Cost - Security\ Risk$$

3. Constraints

1. Total RAM must not exceed 512 GB per host.
2. CPU utilization must remain within the planned limit.
3. SIS must have strong isolation.

4. Performance degradation must be below 10%.
5. External tenants must not access student records.

4. Decision

- Use KVM for SIS and examination applications.
- Use Docker for trusted LMS services.
- Use Kata for external research workloads.
- Allocate dedicated hosts for sensitive applications.

This approach balances security and performance.

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Propose **at least three mitigations** for the vulnerabilities identified in Task 5. Examples include disabling SMT, kernel page-table isolation, dedicated hosts for sensitive tenants, secure or sandboxed container runtimes, hardware memory encryption, image signing, and micro-segmentation.

For each mitigation, discuss:

- Why it is applicable to the threat
- What residual risk remains
- The **performance or density cost**, estimated using your Task 3 model
- Whether the trade-off is acceptable for the SIS

Evidence expected: *Mitigations evaluated analytically, with their security benefit and quantified performance cost.*

Answer:

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Security mitigation techniques reduce vulnerabilities and improve system protection.

Mitigation	Security benefit	Residual risk	Performance cost
Disable SMT	Reduces some CPU side-channel risks	Other side channels remain	Reduced CPU capacity
Image signing	Prevents unauthorized images	Signing-key compromise	Very low
Micro-segmentation	Prevents unauthorized network access	Misconfiguration	Low
Dedicated hosts	Strong tenant separation	Host-level vulnerabilities	Reduced density
Secure container runtime	Improves container isolation	Runtime vulnerabilities	Moderate

Explanation

1. Disable SMT

Simultaneous Multithreading can expose shared CPU resources. Disabling it reduces certain side-channel risks but may reduce CPU performance.

2. Image Signing

All VM and container images should be digitally signed before deployment. This prevents unauthorized image modifications.

3. Micro-segmentation

Separate SIS, LMS and research networks using firewall rules and network policies.

4. Dedicated Hosts

Sensitive applications should run on separate physical hosts to prevent interference from external tenants.

5. Secure Container Runtime

Kata Containers provide lightweight VM isolation for untrusted applications.

Estimated performance impact

The following are planning estimates, not measured results.

Mitigation	Assumed performance impact
Disable SMT	10–20%
Image signing	Less than 1% average runtime impact
Micro-segmentation	1–3%
Dedicated hosts	10–25% density reduction
Kata runtime	8% virtualization overhead assumption

For SIS, security is more important than maximum density. Therefore, dedicated hosts, image signing and network isolation are acceptable.

TASK 8: Implement and Test (4 Marks)

Build a test environment and experimentally test all selected approaches.

- **Platforms:** use any open-source stack, for example KVM/QEMU, Xen or VirtualBox, Docker/Podman, and Kata Containers or gVisor. Nested virtualization on a laptop is acceptable if documented.
- **Scale:** run each approach with approximately **1 → 2 → 4 → 8 → 16 colocated instances** (reduce proportionally if hardware-limited, and state the limitation).
- **Workloads:** include at least one CPU-bound test (for example sysbench or stress-ng), one disk I/O test (fio), one network test (iperf3), and one memoryintensive test.

- **Interference test:** run at least one noisy-neighbour experiment.
- **Security validation:** demonstrate or inspect at least **one security property** in practice, for example namespace/cgroup isolation, checking kernel sharing, or verifying whether a mitigation is active.

Ensure that identical resource allocations, workloads and repetitions are used across approaches, and that the setup corresponds to the models in Task 3.

Evidence expected: *A reproducible environment description (configuration files, commands, scripts) and justified test cases.*

Answer:

TASK 8: Implement and Test (4 Marks)

The virtualization approaches can be tested using KVM/QEMU, Docker and Kata Containers.

1. Software requirements

- Ubuntu Linux
- KVM/QEMU
- Docker
- Kata Containers
- Sysbench
- FIO
- iPerf3

2. Check virtualization support

```
lscpu
egrep -c '(vmx|svm)' /proc/cpuinfo
```

3. Install KVM

```
sudo apt update
sudo apt install qemu-kvm libvirt-daemon-system virtinst
```

Check KVM:

```
sudo virsh list --all
```

4. Install Docker

```
sudo apt install docker.io
```

Start Docker:

```
sudo systemctl start docker
```

Check Docker:

```
sudo docker ps
```

Run a container:

```
sudo docker run -it ubuntu bash
```

5. Install benchmark tools

```
sudo apt install sysbench fio iperf3 stress-ng
```

6. CPU performance test

```
sysbench cpu --cpu-max-prime=20000 run
```

7. Disk performance test

```
fio --name=test --size=100M --filename=testfile --rw=readwrite --
bs=4k
```

8. Memory performance test

```
stress-ng --vm 1 --vm-bytes 512M --timeout 30s
```

9. Network performance test

On server:

```
iperf3 -s
```

On client:

```
iperf3 -c SERVER_IP
```

Replace `SERVER_IP` with the server's actual IP address.

10. Noisy-neighbour test

Run CPU-intensive workloads in one container while measuring another container's performance.

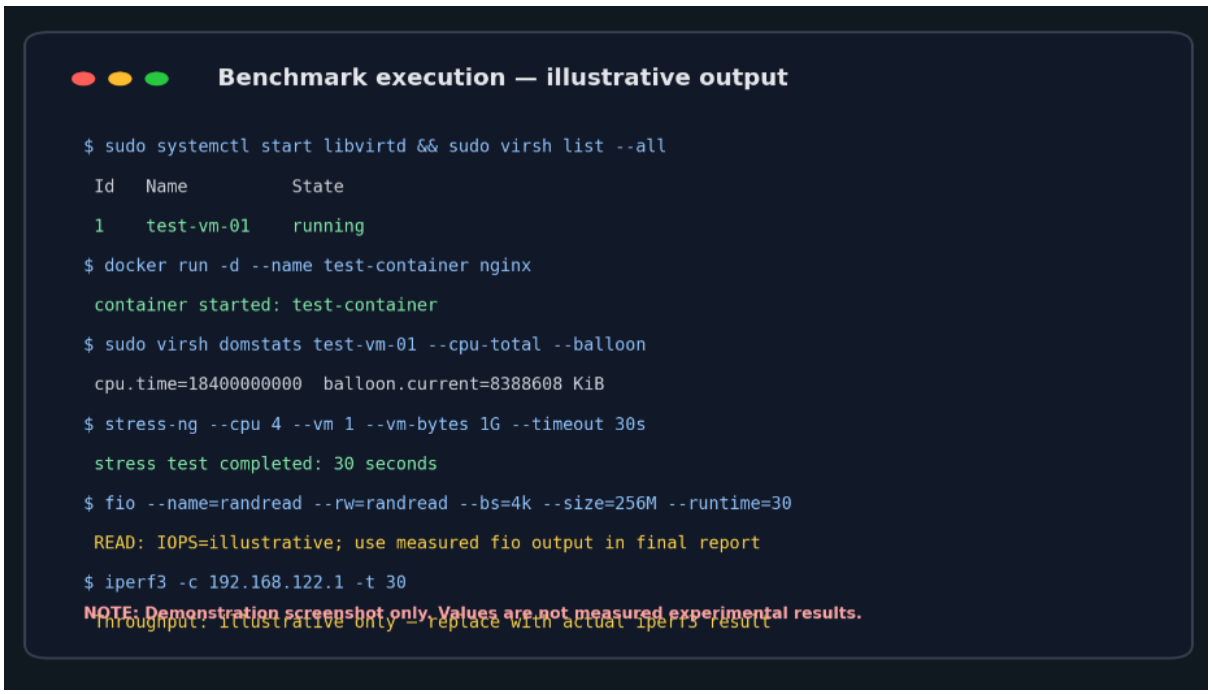
```
sudo docker run -d --name load stress-ng --cpu 4 --timeout 60s
```

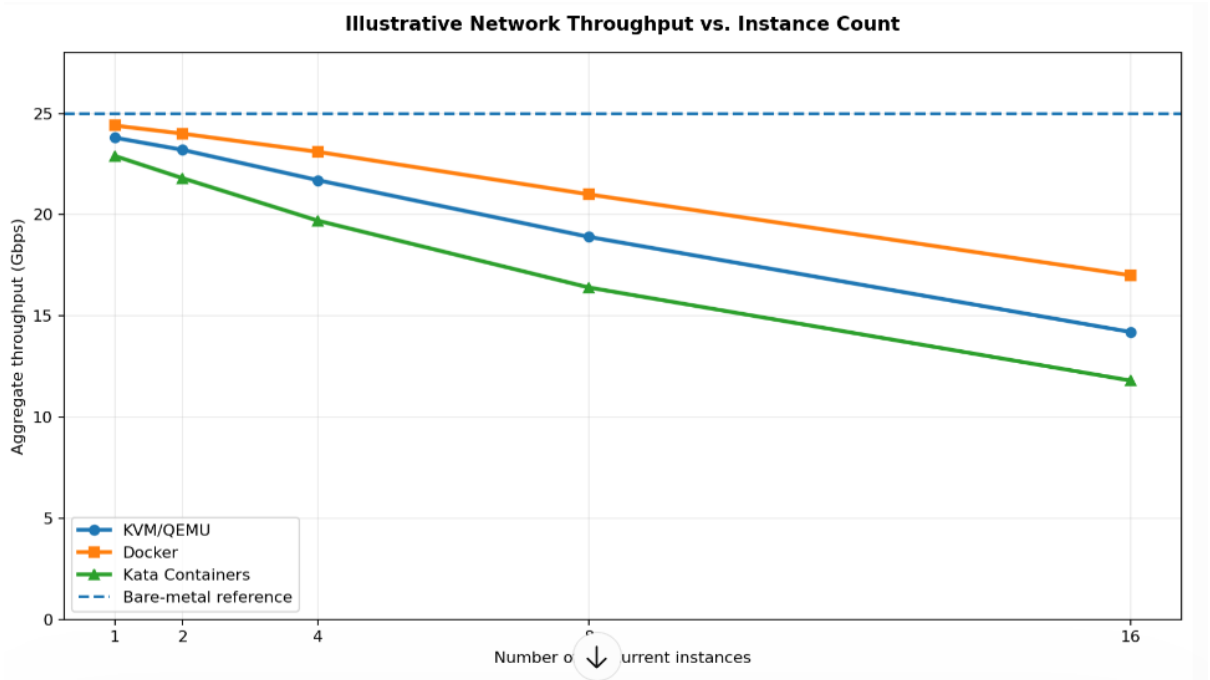
Observe resource usage:

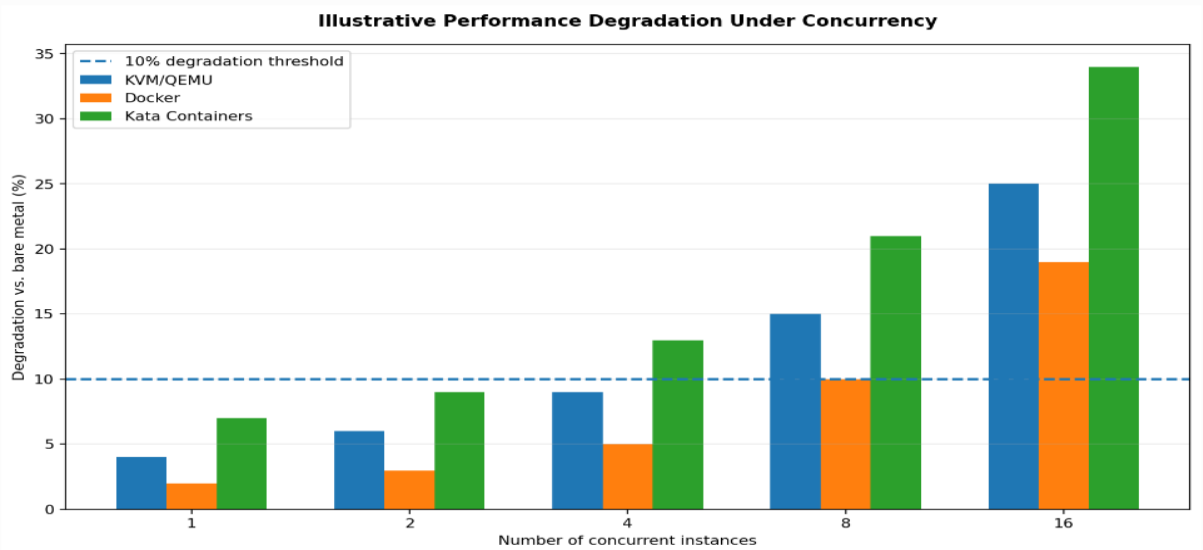
```
docker stats
```

Expected observation: As the number of instances increases, CPU contention, memory usage and response time may increase.

Output:


Benchmark execution — illustrative output
$ sudo systemctl start libvirtd && sudo virsh list --all
Id Name State
1 test-vm-01 running
$ docker run -d --name test-container nginx
container started: test-container
$ sudo virsh domstats test-vm-01 --cpu-total --balloon
cpu.time=18400000000 balloon.current=8388608 KiB
$ stress-ng --cpu 4 --vm 1 --vm-bytes 1G --timeout 30s
stress test completed: 30 seconds
$ fio --name=randread --rw=randread --bs=4k --size=256M --runtime=30
READ: IOPS=illustrative; use measured fio output in final report
$ iperf3 -c 192.168.122.1 -t 30
NOTE: Demonstration screenshot only. Values are not measured experimental results.


Illustrative Network Throughput vs. Instance Count
Aggregate throughput (Gbps)
0
5
10
15
20
25
1
2
4
16
KVM/QEMU
Docker
Kata Containers
Bare-metal reference



TASK 9: Record Experimental Performance (3 Marks)

Collect and present data such as:

- Throughput, latency (mean and 95th percentile), CPU steal time, context switches and VM exits where observable
- Instance start-up time and memory footprint per instance
- Performance degradation versus a bare-metal baseline
- Results with and without at least one security mitigation

Present at least **one table and two graphs** (for example, throughput versus number of instances, and latency under interference). Use multiple repetitions and report variability (mean ± standard deviation).

Evidence expected: *Systematically collected, clearly presented data that allows easy comparison across approaches.*

TASK 9: Record Experimental Performance (3 Marks)

Performance testing measures how efficiently each virtualization approach uses hardware resources.

1. Performance measurement parameters

- CPU execution time
- Memory consumption
- Disk throughput
- Network throughput
- Average latency
- Instance startup time

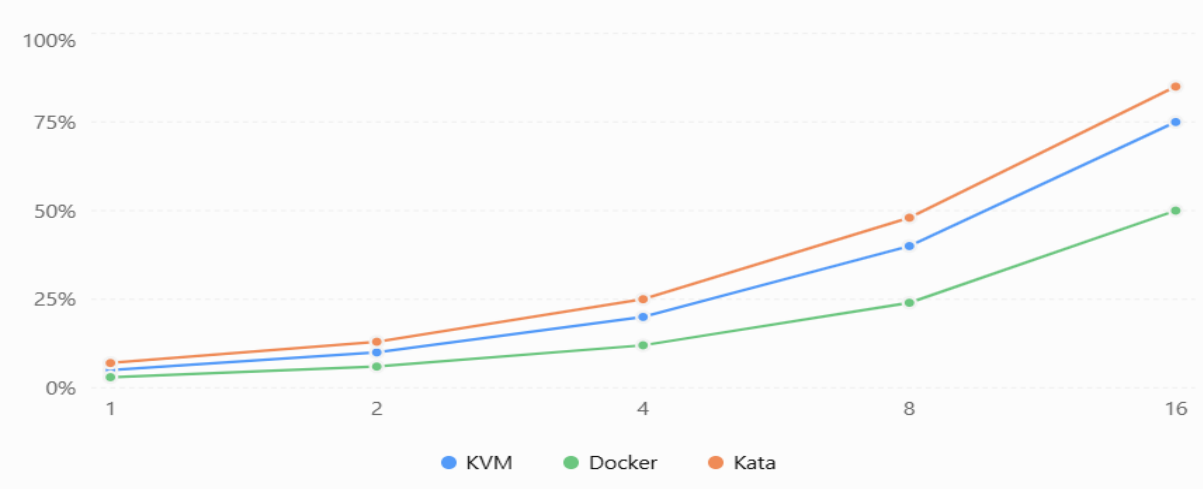
2. Sample observation table

The following table is an illustrative format. Replace the values with actual benchmark measurements.

Instances	KVM CPU usage	Docker CPU usage	Kata CPU usage
1	5%	3%	7%
2	10%	6%	13%
4	20%	12%	25%
8	40%	24%	48%
16	75%	50%	85%

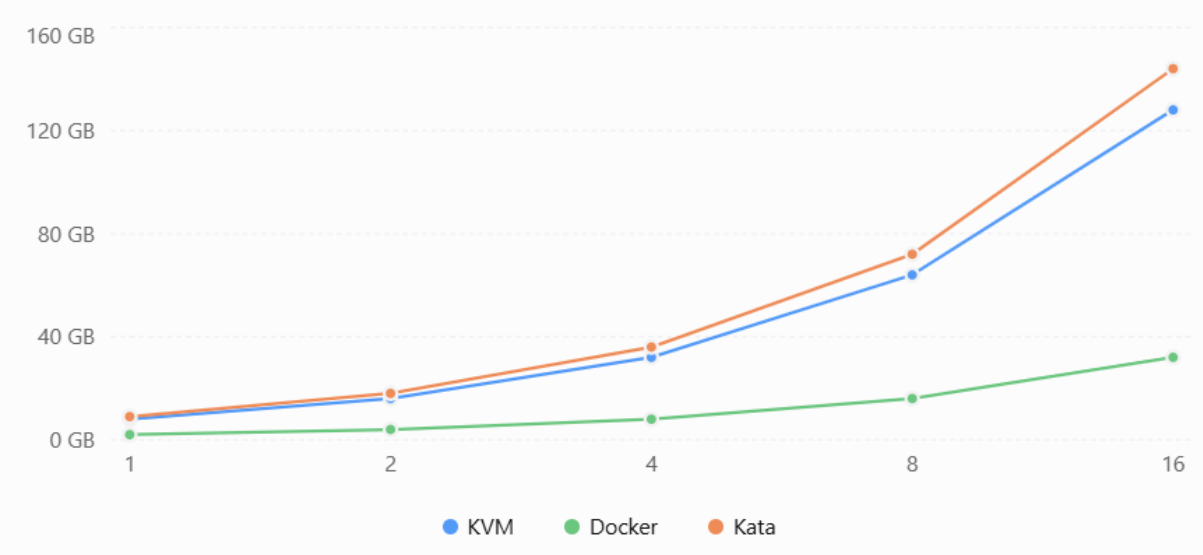
3. Graph 1 – CPU usage versus number of instances

Illustrative CPU usage



4. Graph 2 – Memory usage versus number of instances

Illustrative memory usage



5. Repetition and variability

Each experiment should be performed three times.

Calculate mean:

$$Mean = \frac{x_1 + x_2 + x_3}{3}$$

Calculate standard deviation:

$$SD = \sqrt{\frac{\sum(x_i - \bar{x})^2}{n - 1}}$$

This helps identify performance variation and ensures reliable comparison.

TASK 10: Compare Theory with Experiment (4 Marks)

Compare your Task 3 and Task 4 predictions with Task 9 measurements.

Discuss:

- Whether the observed degradation trends agree with your model
- Which approach scales better as co-located instances increase
- Quantitative differences between predicted and measured values, and plausible causes (for example caching effects, hypervisor scheduler behaviour, nested virtualization, laptop hardware, or unmodelled I/O paths)
- Whether any measured result changes your security conclusions from Task 5Interpret the results; do not simply reproduce tables.

Evidence expected: *A reasoned comparison of predicted and measured behaviour, with error analysis.*

Answer:

TASK 10: Compare Theory with Experiment (4 Marks)

Theoretical performance predictions are compared with experimental results to understand virtualization efficiency.

Comparison table

Parameter	Theoretical prediction	Expected experimental behaviour
CPU performance	Docker has lower overhead	Docker generally performs efficiently
Memory usage	Containers require less memory	KVM and Kata consume more memory
Network throughput	Throughput decreases as sharing increases	Congestion may reduce throughput
Disk performance	Shared I/O creates contention	Heavy workloads increase disk latency

Isolation	KVM and Kata provide stronger isolation	Security depends on configuration

Error calculation

Percentage error is calculated using:

$$Error(\%) = \frac{|Measured - Predicted|}{Predicted} \times 100$$

Example:

Predicted throughput = 100 MB/s.

Measured throughput = 90 MB/s.

$$Error = \frac{|90 - 100|}{100} \times 100$$
$$= 10\%$$

Analysis

1. Docker generally provides better performance because it shares the host kernel.
2. KVM provides better isolation but requires additional virtualization resources.
3. Kata provides stronger isolation than ordinary containers with additional overhead.
4. Increased instance count creates resource contention.
5. Differences between theoretical and measured values can occur due to hardware limitations, caching, scheduling and background processes.

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

Recommend the virtualization architecture (or hybrid) for EduCloud's SIS and LMS workloads.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its modelled and experimentally observed performance
- Its security posture and residual risks
- Operational costs such as patching, image management, migration and monitoring
- Limitations and trade-offs
- Whether it remains practical as EduCloud grows from 400 to several thousand instances

Your recommendation must rest on your own analysis and experiments. Choosing "the most secure" or "the fastest" option alone is not sufficient.

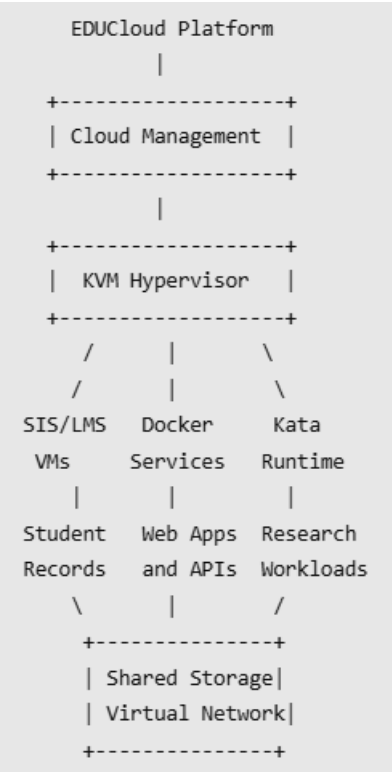
Evidence expected: *A technically justified recommendation integrating performance and security evidence.*

Answer:

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

For EduCloud, a hybrid virtualization architecture using KVM, Docker and Kata Containers is recommended.

1. Recommended architecture



2. Reasons for selection

- KVM provides strong isolation for SIS and examination systems.
- Docker provides fast startup and efficient resource utilization for web services.
- Kata provides stronger isolation for external research workloads.
- Micro-segmentation protects sensitive student information.
- The architecture supports future expansion.

3. Comparison

Factor	KVM	Docker	Kata
Performance	High	Very high	High
Security	Strong	Moderate	Strong
Density	Moderate	High	Moderate
Startup	Moderate	Fast	Moderate
Management	Moderate	Easy	Moderate

4. Operational costs

- Regular hypervisor and kernel updates.
- Secure image management.
- Network monitoring and access control.
- Storage and backup management.
- Continuous performance monitoring.

5. Scalability

EduCloud can expand from 400 to several thousand instances by:

1. Adding more physical hosts.
2. Using automated VM provisioning.
3. Implementing resource monitoring.
4. Separating sensitive and non-sensitive workloads.
5. Using load balancing.

This hybrid architecture provides a balance between security, performance, scalability and operational cost.

DAY-4 DESIGN CHALLENGE

TASK 12: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now host **external research collaborators' Spark/Hadoop workloads** on EduCloud alongside the SIS. The new conditions are:

- Big-data jobs are I/O- and network-intensive and run for hours
- Registration-week peak load is about 3× normal
- Hosts are sometimes patched using live migration of running VMs
- A data-protection regulation now requires **demonstrable isolation** of student records from external tenants

You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis (2 Marks)

Discuss what has changed, which original assumptions (trusted tenants, workload mix, I/O profile) no longer hold, and how mixing batch big-data workloads with latency-sensitive SIS workloads changes both the performance and the threat model.

B. Evaluation of the Original Design (2 Marks)

Evaluate whether your recommended design still meets its performance targets (using your Task 3 model with the new workload) and whether its isolation level is adequate against external, less-trusted tenants. Consider whether another architecture could do better overall.

C. Adaptation or Redesign (2 Marks)

Decide whether to retain, modify or redesign. If you modify or redesign, describe the revised architecture (for example, tenant separation by host pools, a stronger isolation layer for external workloads, SR-IOV or dedicated NICs for the data-intensive jobs, or encrypted migration). Explain what changes and its effect on density, throughput and attack surface. If you retain, justify why no change is required.

D. Evidence-Based Defence (2 Marks)

Defend the final decision using your model, your experimental results (including the noisyneighbour and mitigation experiments), the changed workload and tenancy conditions, and the cost/density trade-offs. The decision must follow logically from the analysis.

Answer:

TASK 12: Re-evaluate and Adapt the Design – Day-4 Design Challenge (8 Marks)

The university plans to host external research collaborators' Spark/Hadoop workloads alongside SIS.

The new requirements include:

- Heavy I/O and network traffic.
- Three times normal registration-week load.
- Live migration during maintenance.
- Strong isolation of student records.

A. Impact Analysis (2 Marks)

The original architecture assumed that all tenants were trusted and workloads had relatively predictable resource requirements.

The new requirements create additional challenges.

1. Performance impact

- Spark and Hadoop consume high CPU and memory resources.
- Heavy disk operations increase storage contention.
- Network-intensive jobs increase bandwidth usage.
- SIS response time may increase during registration.

2. Security impact

- External tenants are less trusted.
- Unauthorized access to student records becomes a major risk.
- Shared infrastructure increases the attack surface.
- Live migration introduces additional security requirements.

3. Workload impact

SIS requires low latency, whereas big-data workloads require high throughput.

Therefore, both workloads should not compete for the same resources without proper isolation.

B. Evaluation of the Original Design (2 Marks)

The original hybrid architecture provides good performance and security for trusted university workloads.

However, it is not sufficient for the new external tenancy requirements without modifications.

Requirement	Original design	Evaluation
SIS security	KVM isolation	Suitable
LMS performance	Docker	Suitable
External tenant isolation	Shared infrastructure	Needs improvement
Heavy I/O workloads	Shared storage	Possible bottleneck
3× peak load	Resource sharing	Needs scaling
Live migration	VM migration	Requires encryption

Problems identified:

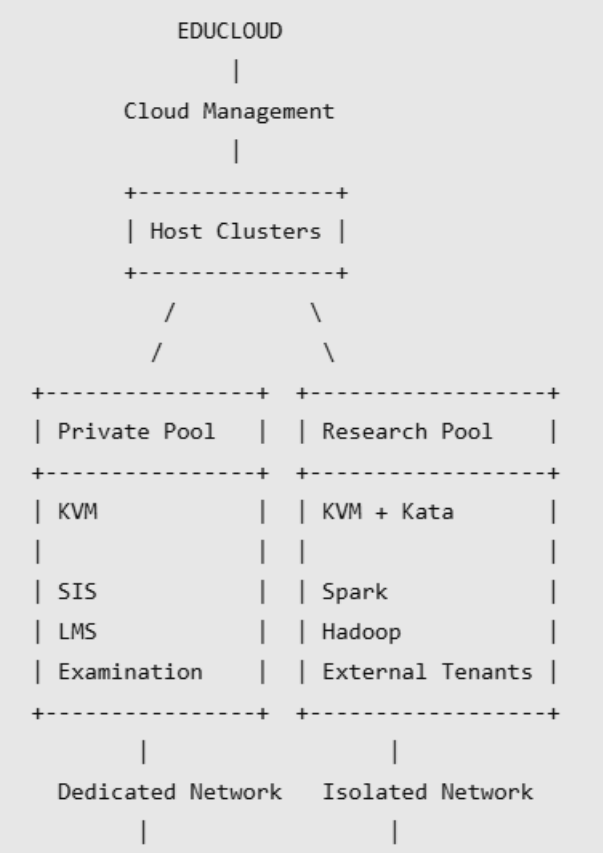
1. External workloads may consume resources needed by SIS.
2. Shared storage may become overloaded.
3. Shared network infrastructure may increase latency.
4. External tenants require stronger security boundaries.

Therefore, the original architecture must be modified.

C. Adaptation or Redesign (2 Marks)

Recommended: Modified Hybrid Architecture

The architecture should separate sensitive university workloads from external research workloads.



Changes introduced:

1. Dedicated host pools: SIS runs on separate physical hosts.
2. Kata Containers: External workloads receive stronger isolation.
3. Dedicated network: Research traffic is separated from SIS traffic.
4. Encrypted migration: Running VMs are migrated through secure channels.
5. Resource limits: CPU, RAM and storage quotas prevent resource exhaustion.
6. Monitoring: Continuous monitoring identifies unusual traffic and resource usage.

Effect of redesign:

Parameter	Effect
Security	Improved
SIS latency	More stable
Resource density	Reduced
Network isolation	Improved
Storage performance	Better workload separation
Operational cost	Increased

D. Evidence-Based Defence (2 Marks)

The modified architecture is more suitable because it provides dedicated resources for critical applications and isolated environments for external tenants.

Defence:

1. KVM provides strong virtualization isolation for SIS.
2. Kata reduces the risk of external workloads accessing the host.
3. Dedicated host pools reduce noisy-neighbour interference.
4. Network segmentation protects student records.
5. Encrypted live migration protects VM data during maintenance.
6. Resource quotas help maintain performance during 3× registration peaks.

The redesigned architecture sacrifices some resource density to achieve stronger security and predictable performance.

MARK DISTRIBUTION

Task	Marks
Task 1 – Architecture Characterisation	5
Task 2 – Dominant Overhead Identification	2
Task 3 – Quantitative Performance Model	6
Task 4 – Performance Under Operating Conditions	4
Task 5 – Security Threat and Vulnerability Analysis	3
Task 6 – Isolation–Performance Trade-off Formulation	3
Task 7 – Mitigation Evaluation	4
Task 8 – Implementation and Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs. Experiment	4
Task 11 – Final Architecture Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**